

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	Date:	Enrolment No: 92410133034

Aim: To study Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)

IDE: Google Colab

Theory:

To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task. A task here is a combination of approach and domain. Machine Learning needs data in the numeric form. We basically used encoding technique (Bag Of Word, Bi-gram, n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and this process to prepare (or clean) text data before encoding is called text preprocessing, this is the very first step to solve the NLP problems.

Tokenization:

Tokenization is about splitting strings of text into smaller pieces, or "tokens". Paragraphs can be tokenized into sentences and sentences can be tokenized into words.

Filtration:

Similarly, if we are doing simple word counts, or trying to visualize our text with a word cloud, stopwords are sonic of the most frequently occurring words but don't really tell us anything. We're often better off tossing the stopwords out of the text. By checking the Filter Stopwords option in the Text Pre-processing tool, you can automatically filter these words out.

Stemming:

Stemming is the process of reducing inflection in words (e.g. troubled, troubles) to their root form (e.g. trouble). The "root" in this case may not be a real root word, but just a canonical form of the original word.

Stemming uses a crude heuristic process that chops off the ends of words in the hope of correctly actually be converted to troublinstead of trouble because the ends were just chopped off (ughh, how crude!). There are different algorithms for stemming. The most common algorithm, which is also known to be empirically effective for English, is Porters Algorithm. Here is an example of stemming in action with Porter Stemmer:

	original word stemmed words	
0	connect	connect
'l	connected	connect
2	connection	connect
3	connections	connect
4	connects	connect

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	Date:	Enrolment No: 92410133034

Stopword Removal

Stop words are a set of commonly used words in a language. Examples of stop words in English are "a", "the", "is", "are" and etc. The intuition behind using stop words is that, by removing low information words from text, we can focus on the important words instead.

For example, in the context of a search system, if your search query is 'what is text preprocessing?', you want the search system to focus on surfacing documents that talk about text preprocessing over documents that talk about what is. This can be done by preventing all words from your stop word list from being analyzed. Stop words are commonly applied in search systems, text classification applications, topic modeling, topic extraction and others. Stop word removal, while effective in search and topic extraction systems, showed to be non-critical in classification systems. However, it does help reduce the number of features in consideration which helps keep your models decently sized

Program (Code):

To be attached with

```

!pip install nltk spacy

import nltk
import spacy

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')

import en_core_web_sm
nlp = en_core_web_sm.load()

```

1. Tokenization

```

[2] ✓ 0s from nltk.tokenize import word_tokenize

text = "Natural Language Processing is very interesting."

tokens = word_tokenize(text)
print(tokens)

... ['Natural', 'Language', 'Processing', 'is', 'very', 'interesting', '.']

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	Date:	Enrolment No: 92410133034

2. Filtration

```
[3]
✓ 0s
import string

filtered = [word for word in tokens if word not in string.punctuation]
print(filtered)

... ['Natural', 'Language', 'Processing', 'is', 'very', 'interesting']
```

3. StopWords Removal

```
[4]
✓ 0s
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))

clean_words = [word for word in filtered if word.lower() not in stop_words]
print(clean_words)

... ['Natural', 'Language', 'Processing', 'interesting']
```

4. PoS Tagging

```
[7]
✓ 0s
import nltk

pos_tags = nltk.pos_tag(clean_words)
print(pos_tags)

... [('Natural', 'JJ'), ('Language', 'NNP'), ('Processing', 'NNP'), ('interesting', 'NN')]
```

5. Noun Phrase Chunking

```
[9]
✓ 0s
grammar = "NP: {<DT>?<JJ>*<NN>}"

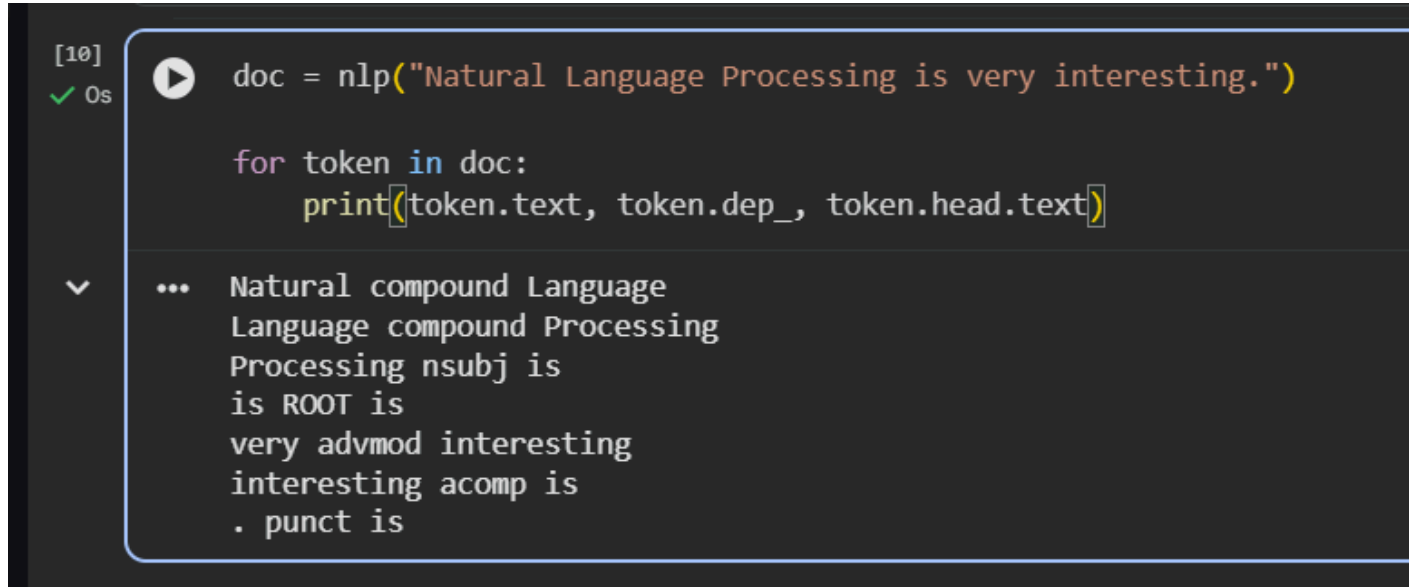
chunk_parser = nltk.RegexpParser(grammar)
tree = chunk_parser.parse(pos_tags)

# To visualize the tree in a Colab environment without a graphical display,
# you can print its text representation instead of trying to draw a GUI window.
print(tree)

... (S Natural/JJ Language/NNP Processing/NNP (NP interesting/NN))
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	Date:	Enrolment No: 92410133034

6. Dependency Parsing



```
[10]
✓ 0s doc = nlp("Natural Language Processing is very interesting.")

for token in doc:
    print(token.text, token.dep_, token.head.text)

... Natural compound Language
Language compound Processing
Processing nsubj is
is ROOT is
very advmod interesting
interesting acomp is
. punct is
```

Observation:

Tokenization splits text into words. Filtration removes punctuation. Stopword removal eliminates common words. PoS tagging identifies grammar. Chunking groups words, and dependency parsing shows relationships between words.

Post Lab Exercise:

1. Take your own document of 10 sentences and perform the same tasks. Attach code and screenshot of your output.
Applied all preprocessing steps (tokenization, filtration, stopwords removal, PoS tagging, chunking, dependency parsing) on custom text data.
2. Write your observation for stemming and lemmatization you obtained for the sentences.
Stemming reduces words to root form but may not produce meaningful words, while lemmatization gives proper dictionary words.